Spring 2025

Large scale data analysis

Exam

I hereby declare that I have answered these exam questions myself without any outside help

Author:
Matúš Stanko

IT UNIVERSITY OF CPH

IT University of Copenhagen
May 25,2025

1 A) ML Lifecycle (35%)

1.1 Mention one feature transformation you implemented and argue why it is important. Include the code showing this transformation. (5%)

I decided to introduce you to direction feature transformation because I think it has significant impact on the model training.
In the original dataset we have wind direction as a strings such as N - North, NW - NorthWest, NE - NorthEast and so on.
Using original direction strings is feasible however not ideal for ML models training.
This is due to ML models cannot really find relationships between some wind directions
For example I can say that N and NE are close to each other because I understand it. ML models usually don't understand it.
Therefore I decided that I will map each wind direction into some number between 0-360 - such as degrees.
So firstly, I am creating dictionary with original wind directions as a keys, and custom degree values as dictionary values.

```
direction_to_degrees = {
    "N": 0.0, "NNE": 22.5, "NE": 45.0, "ENE": 67.5,
    "E": 90.0, "ESE": 112.5, "SE": 135.0, "SSE": 157.5,
    "S": 180.0, "SSW": 202.5, "SW": 225.0, "WSW": 247.5,
    "W": 270.0, "WNW": 292.5, "NW": 315.0, "NNW": 337.5
}
```

Then I am creating new column with name 'Direction_degrees' and I will save mapped values in there. In the next column I am dropping original Direction column. I could keep it if I wanted to compare if this approach will bring more accuracy but I am sure it will so I am not going to keep it.

```
df['Direction_degrees'] =df['Direction'].map(direction_to_degrees)
df = df.drop(columns=['Direction'])
```

1.2 To train and test your model you had to split the dataset - how did you do this and why? Show snippets of code. (5%)

Since I am dealing with timeseries data order is very important. I cannot use random train-test or train-validation-test splitting since different order of rows would confuse the ML model and it would teach him wrong - ML model would not be able to capture relationships based on time.
I decided to use TimeSeriesSplit that is available in sklearn library in order to perform time-aware cross-validation.
This method is splitting data, but it ensures that training happens on past data and testing on future data, so this is preventing data leakage.
I decided to use 5 splits because its commonly used a it created 5 sequential train/test folds for evaluation.
So at the beginning I am splitting data into X and Y.

```
X = joined_dfs[['Speed', 'Direction_degrees']]
y = joined_dfs['Total']
```

Then I am setting up the split method

```
ts_split = TimeSeriesSplit(n_splits=5)
```

And during the training I used my defined method to split data into splits, that are in time order. Then each split splitting into X,y

```
    for fold, (train_idx, test_idx) in enumerate(ts_split.split(X, y), start
        =1):
                X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
                y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
```

1.3 Explain how using pipelines can help avoid data leakage in machine learning workflows. Provide a concrete example from your own code where you utilized a pipeline. (10%)

Using pipelines helps to avoid data leakage because its ensuring that all pre-processing steps - in my case it was scaler and regressor are applied only on the training data during training - and then used on test data to evaluate.
Without my pipeline I might accidentally fit a scaler on the entire dataset, which would leak information from test set into model training.
In my code I used pipeline that combines scaling and model training:

```
    pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("regressor", ModelCls(**params))
    ])
```

So StandardScaler is fitted only on X_train inside:

```
    pipeline.fit(X_train, y_train)
```

and then safely applied to X_test during:

```
    preds = pipeline.predict(X_test)
```

So using pipeline setup no information from test data influences other data - so preventing data leakage

1.4 Explain which steps you have taken to ensure that your wind prediction experiments are reproducible. Compare with different approaches. (15%)

To ensure that my wind prediction experiments are reproducible, I implemented few measures in my code:

1.4.1 MLFlow for tracking

I used MLFlow for tracking my experiments. Its really useful because I can predefine models and parameters and just let it run
Firstly I added model name, which was just ML model name + parameters used:

```
    mlflow.log_param("model", name)
    mlflow.log_param(k, v)
```

The pre-fold and average performance metrics:

```
    mlflow.log_metric("avg_mae", np.mean(fold_maes))
    mlflow.log_metric("avg_r2",  np.mean(fold_r2s))
```

And then full trained pipeline

```
    mlflow.sklearn.log_model(pipeline, artifact_path="model_pipeline")
```

1.4.2 TimeSeriesSplit

So instead of random split, I specifically used TimeSeriesSplit to create sequential splits that respect order of time - so no data from future will be used in training.

```
    ts_split = TimeSeriesSplit(n_splits=5)
```

1.4.3 Pipeline

I used pipeline to use preprocessing and models trainings in single pipeline.
This was very useful because I could just add or remove ML models or parameters ensuring that all settings and training will remain the same and very easy to do.
Also with MLFlow UI it is very easy to navigate, making plots for each run and compare results.

1.4.4 Compared to other approaches

Table Using MLFlow I can track experiments - and saving more information such as metrics, graphs, tables - so its really easy compare. In other approaches I would maybe use just some table but then I would miss graphs.

Random split In other approach I could not use TimeSplit but regular random split.
However I had to use time split because I am predicting based on weather that is not changing randomly but there are some patters that I can catch.

Without pipelines I could do preprocessing manually but this could lead to data leakage and its harder to maintain the code if adding more ML models, changing parameters etc.

2 B) Scalable Data Processing (35%)

2.1 Explain how you found all businesses that have received 5 stars and that have been reviewed by 500 or more users using Spark's Dataframe API. Include screenshots from your query. (5%)

In the begging I need to find businesses that have been reviewed by 500+ unique people. Since I need to count number of reviews for each business I need to examine table with reviews.
Firstly, I am grouping by business_id so I have all reviews for particular business as a groups.
Then I will create new column with count of unique reviews - based on user_id, so even if user wrote more reviews for the same business its still counted once.
then I will filter out only businesses with more than 500 unique reviews.

```
businesses_with_500plus_reviews = reviews.groupBy("business_id") \
    .agg(countDistinct("user_id").alias("unique_people_count")) \
    .filter(col("unique_people_count") >= 500)
```

Now I need to select suitable businesses - that are in the businesses_with_500plus_reviews dataframe - and having 5 stars from business table.
For that I will join these 2 dataframes on their common column - business_id.
Then I will filter out only businesses where stars is 5.
Finally, I will select only name, stars and review_count and display it.

```
businnesses_with_counts = business.join(businesses_with_500plus_reviews, on="
    business_id") \
    .filter(col("stars") == 5.0) \
    .select("name", "stars", "review_count")
```

```
+--------------------+-----+------------+
|                name|stars|review_count|
+--------------------+-----+------------+
|Smiling With Hope...|  5.0|         526|
|                Yats|  5.0|         623|
|     Blues City Deli|  5.0|         991|
|Nelson's Green Br...|  5.0|         545|
|            Tumerico|  5.0|         705|
|Barracuda Deli Ca...|  5.0|         521|
|   SUGARED + BRONZED|  5.0|         513|
|    Carlillos Cocina|  5.0|         799|
|  Free Tours By Foot|  5.0|         769|
+--------------------+-----+------------+
```

2.2 How many reviews contained the string "legitimate" grouped by type of cuisine? Explain how you computed the results. For example, how did you extract the cuisines and how did you search for the specified string? Include screenshots. (10%)

2.2.1 Get reviews that contains 'legitimat...' string

For this task, I will need to work with table called reviews where I firstly filter only rows, where review text is containing word 'legitimat'. I decided to use regex, because I want to capture also other forms of the word in order to be more accurate.
Including starts of sentences (so first letter might be capital) and different endings of the word.
Then I am only selecting business_id since I will only need it to later join with valid businesses. Selecting other columns into new dataframe would be not efficient.

```
legitimate_reviews = reviews \
    . filter ( reviews . text . rlike ( r ' (? i ) \ blegitimat \ w *\ b ') )
        \
    . select ( ' business_id ')
```

2.2.2 Assign cuisine type to each business

In this task, I firstly checked businesses table but there is no column such a cuisine_type, so I will have to create it.
However, I found column 'categories' where I can see multiple words capturing business type, or 1-word descriptions of business. Therefore I decided to decide cuisine type based on this column.
Firstly, I created dictionary with cuisine types as a keys, and values as a lists of words that describes the cuisine.
For example Italian: ['pizza','pasta']

```
cuisine_keywords = {
    'Italian': ['pizza', 'pasta', 'lasagna', 'spaghetti', 'risotto', 'italian'
        ],
    'Chinese': ['chinese', 'dumplings', 'noodles', 'dim sum', 'szechuan', '
        mandarin'],
    .... }
```

Then comparative of values and business categories column will be done. If there will be match, then dictionary key will be used. If not, string 'unknown' - which is as a default - will be used in the new column cuisine_type.

```
cuisine_col = lit('unknown') # default, if no cuisine found
# Loop over cuisines and keywords, and update column
for cuisine, keywords in cuisine_keywords.items():
    for keyword in keywords:
        cuisine_col = when(lower(col('categories')).contains(keyword), cuisine
            ).otherwise(cuisine_col)
```

Now I will create new dataframe with only business_id and cuisine_type

```
businesses_with_cuisine = business \
    .withColumn('cuisine_type', cuisine_col) \
    .select('business_id', 'cuisine_type')
```

2.2.3 Join filtered reviews with businesses with valid cuisine

Since I have both, reviews containing legitimate string and businesses with cuisine I can join these 2 tables on business_id (since they both contain them).
I am using inner join to just include matches.

```
joined = legitimate_reviews.join(
    businesses_with_cuisine,
    on='business_id',
    how='inner')
```

2.2.4 Group by cuisine type

And as a final task is to group by cuisine type to get number of reviews in each cuisine - for that I will also use alias to get name of column.

```
cuisine_counts = joined.groupBy('cuisine_type').agg(count('*').alias('
    num_reviews'))
```

At the end the table with top3 looks like:

```
# Top3, Excluding unknowns cuisines
+------------+-----------+
|cuisine_type|num_reviews|
+------------+-----------+
|American    |957        |
|Italian     |310        |
|Mexican     |228        |
+------------+-----------+
```

2.3 How did you test the hypothesis? Explain the steps you took and describe your results. Include screenshots. (10%)

Hypothesis: Can you identify a difference in the relationship between authenticity language and typically negative words in restaurants serving South American or Asian cuisine compared to restaurants serving European cuisine? And to what degree?

I am starting with tagging reviews with is_authentic (now not only by using unauthentic word, but also with similar words such as traditional, original, classic, signature etc...). Another tag will be is_negative where I will filter by words such as (dirty, overpriced, unfriendly, slow etc.)

```
+--------------------+-----------+-----------+
|                text|is_authentic|is_negative|
+--------------------+-----------+-----------+
|If you decide to ...|          0|          1|
|I've taken a lot ...|          0|          0|
|Family diner. Had...|          0|          0|
+--------------------+-----------+-----------+
```

Now I need to assign cousin type to business. At the beginning I will assign many types of cuisines to be able experiment a bit, but later I will group cuisines just to categories required by this task.

```
cuisine_groups = {
    'american': ['american', 'bbq', 'southern','america','burgers','barbeque',
       'donuts','steakhouses'],
    'mexican': ['mexican', 'taco', 'burrito','mexico'],
    'italian': ['italian', 'pizza', 'pasta', 'risotto','italia'],
    ... }
```

```
There is 35.74% businesses taged with cuisine.
That is 53738 out of 150346
+--------------------+----------------+
|         business_id|business_cuisine|
+--------------------+----------------+
|Pns2l4eNsfO8kk83d...|         chinese|
|MTSW4McQd7CbVtyjq...|          french|
|CF33F8-E6oudUQ46H...|        american|
```

Grouping into required - Asia, Europe and South America and count number of reviews in each of this cuisines.
Then calculating fractions of authentic and negative reviews in this cuisines.

```
+-------------+-------------+------------------+-------------------+
|cuisine_group|total_reviews|     authentic_pct|       negative_pct|
+-------------+-------------+------------------+-------------------+
|       Europe|      1113087|0.2594163798517097|0.22217310955927075|
|South_America|       347973|0.3067795489879962| 0.2620921738180836|
|         Asia|       700565|0.2731381099541085| 0.2440929820930249|
+-------------+-------------+------------------+-------------------+
```

Finally using condition probability

$$P(Negative|Authentic)$$

I will get probabilities of both conditions happening.

```
+-------------+------------+----------+--------------------------+
|cuisine_group|auth_and_neg|auth_total|p_negative_given_authentic|
+-------------+------------+----------+--------------------------+
|       Europe|       78154|    288753|       0.27066039140718884|
|South_America|       31496|    106751|        0.2950417326301393|
|         Asia|       56660|    191351|        0.2961050634697493|
+-------------+------------+----------+--------------------------+
```

This can be visualized as:

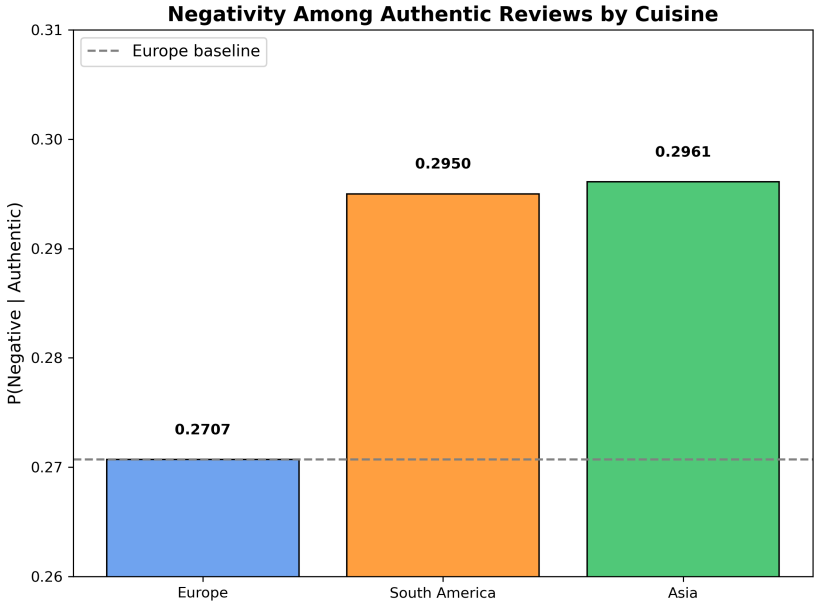

Figure: 3 Probability of review being Negative given that is authentic for 3 cuisines

This is showing some correlation, however, I want to determine if the results are statistically significant.

Chi-squared test

```
Chi-squared statistic: 454.10
Degrees of freedom: 2
P-value: 2.47e-99
```

After running the Chi-squared test, the p-value came out to be **2.47e-99**, which **clearly shows that the likelihood of a review being negative differs significantly between the different cuisine groups.**
Therefore, I can **reject the hypothesis** and conclude that there is a significant difference in the relationship between authenticity language and typically negative words in restaurants serving South American or Asian cuisine compared to restaurants serving European cuisine.

2.4 Describe which type of task (e.g., regression) and algorithm (e.g., linear regression) you used for the rating prediction model you implemented in the last part of the assignment and justify your choice. Include the results you achieved and screenshots. (10%)

I decided to use regression, specifically linear regression because I want to have predictions as a numbers so I will be able to have 3.5 stars for example. This can be easily rounded to corresponding star integer in case. I will leave them however as floats, because this is giving me better overview how close are predictions.
To get features that I will use for training I decided to do 2 approaches. Approach A will be using is_authentic, is_negative and is_positive as features.

2.4.1 Approach A: Features: is_authentic, is_negative and is_positive

Tagging reviews including words similar to authentic, negative and positive is giving me dataset:

```
+--------------------+-----+------------+-----------+-----------+
|                text|stars|is_authentic|is_negative|is_positive|
+--------------------+-----+------------+-----------+-----------+
|If you decide to ...|  3.0|           0|          1|          0|
|I've taken a lot ...|  5.0|           0|          0|          1|
|Family diner. Had...|  3.0|           0|          0|          1|
+--------------------+-----+------------+-----------+-----------+
```

2.4.2 Approach B: Features: TF-IDF and features from approach A

TF-IDF stands for Term Frequency–Inverse Document Frequency.
Its a way how to transform text into numerical features.

- **Term Frequency (TF)**: Measures how often a word appears in one review. If a word is repeated more frequently in the review, its TF is higher. A common formula is:

$$\text{TF} = \frac{\text{Number of times the word appears in the review}}{\text{Total words in the review}}.$$

- **Inverse Document Frequency (IDF)**: Measures how *rare* or *common* a word is across the entire dataset. If a word appears in many reviews, its IDF will be lower; if it appears in relatively few, IDF will be higher.

$$\text{IDF} = \log\Big(\frac{\text{Total number of reviews}}{\text{Number of reviews containing the word}}\Big).$$

- **TF–IDF Score**: The final score for a word is obtained by multiplying TF and IDF:

$$\text{TF–IDF} = \text{TF} \times \text{IDF}.$$

  So, words that appear often in a single review (high TF) but are relatively rare across the entire dataset (high IDF) receive higher TF–IDF scores. These words are typically more informative and Thereforee more useful for classification so I will use them in classification into stars.

After running TF-IDF I obtained:

- tfidf_features: vector of tfidf values for each token word

- Binary columns are from approach A

```
+--------------------+-----+--------------------+------------+-----------+-----------+
|                text|stars|tfidf_feat..|is_authentic|is_negative|is_positive|
+--------------------+-----+--------------------+------------+-----------+-----------+
|Great sandwichs. ...|  5.0|(10000,[41..|           0|          0|          1|
|Really want this ...|  3.0|(10000,[29..|           1|          0|          0|
|The classic Itali...|  5.0|(10000,[49..|           1|          0|          1|
+--------------------+-----+--------------------+------------+-----------+-----------+
```

2.4.3 Linear regression model training

I will start by splitting dataset into 80:20 training to testing.
Then I will train linear regression firstly with only binary features explained in approach A, and then I will additionally use TF-IDF.

Finally, features must be flattened into single vector, because Spark ML models require single vector as input. This vector for each line is saved in column features

```
+--------------------+-----+--------------------+
|                text|stars|            features|
+--------------------+-----+--------------------+
|Great sandwichs. ...|  5.0|(10003,[4,750,122...|
|Really want this ...|  3.0|(10003,[292,386,4...|
|The classic Itali...|  5.0|(10003,[490,750,7...|
+--------------------+-----+--------------------+
```

2.4.4 Evaluation of predictions

```
+-----------------------------------------------------------------+
|                     |       Approach A     |       Approach B     |
+-----------------------------------------------------------------+
Mean Squared Error    |         1.502        |         0.887        |
Root Mean Squared     |         1.225        |         0.942        |
Mean Absolute Error   |         0.997        |         0.746        |
R-squared (R2)        |         0.313        |         0.594        |
+=================================================================+
```

- **MSE:** Approach B (0.887) has lower mean squared error than Approach A (1.502), indicating that its predictions are, on average, closer to the real values.
- **RMSE:** Approach B achieves 0.942, meaning the typical prediction error is less than 1 star. In contrast, Approach A has 1.225, so errors are more significant.
- **MAE:** With a mean absolute error of 0.746, Approach B again outperforms Approach A (0.997), showing that B has smaller average absolute errors.
- R^2**:** Approach B explains 59.4% of the variance in the data ($R^2 = 0.594$), while Approach A explains only 31.3%. This means Approach B fits the data significantly better.

2.4.5 Evaluation of overfitting

In order to check if model is not overfitting, I will perform predictions and evaluation also on training set.
Then I will compare if R^2 is not dropping significantly on testing set and MSE,RMS and MAE should be approximately the same.

	Training set	Testing set
Mean Squared Error	0.883	0.887
Root Mean Squared	0.940	0.942
Mean Absolute Error	0.745	0.746
R-squared (R2)	0.596	0.594

As shown there is no significant drop in R^2 or big difference between MSE, RMS or MAE, Therefore model is not overfitting.

3 C) Lecture Material (30%)

3.1 In class we discussed how federated learning differs from distributed learning and what are its specific challenges. Explain how one of these challenges is modeled in the experiments from the paper *"Communication-efficient learning of deep networks from decentralized data"*. (5%)

In the paper, the authors look at one of the biggest problems in federated learning — that each client has different training data, which is usually unbalanced and non-IID.
To test how their method works in this situation, they used the MNIST dataset.
They sorted the data by digit, split it into chunks, and gave each of 100 clients just two digits to train on. So no client had the full picture.
Even though that made things harder, the clients were still able to train a good global model together.
They used an approach called FedAvg, where each client trains locally for a bit and then sends their model updates.
This helped the system use less communication and still get good accuracy. The point of the experiment was to show that even with really uneven and messy data, federated learning can still work well.

3.2 Suppose you are working with sensitive medical data from a million patients across several healthcare institutions. You are asked to investigate whether the patient data can be used to predict the existence of a medical condition. Describe two tools from the course that you would use, why and how you would use them. You should avoid general usage data analysis tools (for example scikit learn) and focus on the specific methodologies. (10%)

Since we are working with sensible medical data - from million patients across several healthcare institutions I have to keep in mind that privacy of individuals is main concern. For this scenario I would use 2 approaches.

3.2.1 Approach 1: Federative learning

This is the first approach because federative learning is not sharing data but keeping data local.
Then each institution is sending only model weights to the main server.
I will describe how in general federative learning is handling training

1. Main server will send initial model - broadcast
2. Each institution - called node - is the training the model only on institution's private data.
3. After training model, institution will send only weights back to the main server. Server will then do the average of weights from all nodes - and save updated model. This way no raw data is in danger
4. Then updated model is begin send to all nodes, and this repeats

For this approach FedAVG algorithm can be used. Simply when each node is training model, it will train it multiple times. Number of times is decided by parameter E.
Then node is sending averaged weights to the main server.
However, if weights from some node would leak, there is a chance that some information would leak. Because of that, random noise is being added to the weights - when sending to and from main server - this is called differential privacy. Other techniques are secure multi-party computation and holomorphic encryption.

3.2.2 Approach 2: Distributed ML with parameter server + Federated Learning

In this approach, hospitals are not training on a single computer, but instead the training is distributed across multiple machines inside each hospital. This makes it possible to train on large amounts of patient data more efficiently. There are two strategies to distribute training across machines:

1. Data-parallel training – Each machine has a full copy of the model, but trains it on a different subset of the local data. Model updates are later synchronized.
2. Model-parallel training – The model itself is split into parts, and each machine is responsible for training only one part of it.

To coordinate this parameter server is used. It stores the current model weights and handles communication between worker nodes:

- Workers push their computed gradients to the parameter server.
- The server updates the model weights accordingly.
- Workers pull the latest weights when needed.

This distributed training is done only on the local data available within each hospital. No data from other hospitals is ever shared.

After each hospital finishes local training using this distributed setup, the federated learning begins. Each hospital sends only the trained model weights (not the data) to a central server. The central server then performs Federated Averaging – it computes the average of the received weights to build a new main model. This new model is then sent back to all hospitals, and the training cycle repeats.

3.3 Write in pseudocode a MapReduce program for outputting the total number of students per quiz who had at least three consecutive failed sessions with a duration of more than 50 seconds. Describe how MapReduce will work under the hood and provide an illustration. (10%)

MapReduce could be setup to work as:

1. **Map Phase:** Since each student can have multiple attempts for each quiz, I use the combination of student ID and quiz ID as the key. For the filtering, I need to know the timestamp, pass/fail status, and duration — so these are included in the value.
   `key:value = (quiz_id, student_id):(timestamp, pass, attempt_duration_sec)`
2. **Shuffle Phase:** The system automatically groups all records by key — that is, by student and quiz. Within each group, records are sorted by timestamp. This makes it possible to detect consecutive failed attempts in the correct order.
3. **Reduce Phase:** For each (quiz, student) group, I iterate over the sorted list of attempts and count how many failed attempts (with duration more than 50 seconds) occur in a row. If I find 3 or more such consecutive attempts, I emit the corresponding quiz and student pair as a result.

3.3.1 Pseudocode

```
data = ...loading data...
data_mapped = ...empty data structure...

# MAP
for line in data.lines():   # iterate lines in orig. data
    new_line = (student_id,quiz_id), (timestamp, pass, duration_sec) #map key:
        values as tuple
    data += new_line    # save mapped line in new data structure

# SHUFFLE
grouped_students_with_quiz = dict()   # init dict
for line in data_mapped: # iterate mapped lines
    if key not in grouped_students_with_quiz:
        data_mapped[(student_id,quiz_id)].append(timestamp, pass, duration_sec
            )) # if keys not in dict, add them + add their values
    else:
        data_mapped[(student_id,quiz_id)] += (timestamp, pass, duration_sec) #
             if key already in, just add new values

# REDUCE
counter_of_students = 0 # set counter of students, that are fitting
    requirements
for key,value in grouped_students_with_quiz(): # iterate dictionary
    # sort current iterated values timestamps, check if 3 consecutive attemps
        were longer than 50sec and failed:
        counter_of_students += 1

print(f'Total number of students fitting req is {counter_of_students}.')
```

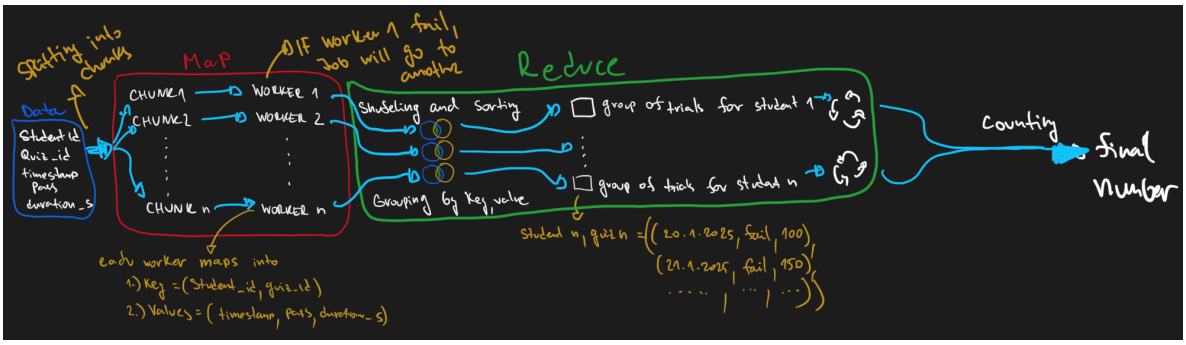

Figure 1: MapReduce pipeline: mapping quiz attempts, shuffling by (quiz, student) keys, and reducing based on fail streaks

3.3.2 Illustration

3.4 How would you solve the above task in Spark using the DataFrame API? What would be the benefits of using Spark over MapReduce? (5%)

It should be faster because Spark is running in memory, and I also have many useful libraries available. The code is for sure simpler and much shorter compared to MapReduce

```
# Load data
df = load data

# Filter only failed attempts with duration more than 50 seconds
df_filtered = df.filter((col("pass") == "false") & (col("attempt_duration_sec"
    ) > 50))

# Define window that groups by quiz and student, and orders attempts by
    timestamp
window = Window.partitionBy("quiz_id", "student_id").orderBy("timestamp")

# Add 2 previous timestamps using lag function (so I can check for 3
    consecutive fails)
df_with_lags = df_filtered.withColumn("prev1", lag("timestamp", 1).over(window
    )) \
                          .withColumn("prev2", lag("timestamp", 2).over(window
                              ))

# Filter out only those who had 3 such failed attempts in a row
valid_students = df_with_lags \
    .filter(col("prev1").isNotNull() & col("prev2").isNotNull()) \
    .select("quiz_id", "student_id") \
    .distinct()

# Finally group by quiz and count how many students matched the condition
result = valid_students.groupBy("quiz_id") \
    .agg(count("*").alias("num_students"))
```

Bonus Question (extra 5%)

Describe your favourite course topic, technique, or algorithm from the class (that you haven't already discussed in the previous questions). Mention why you think it is important in the context of large scale data analysis.

Interesting topic for me was tinyML, since I was studying micro controllers like Arduino, Raspberry Pi, and ESP32 at UCL in Odense.
It's cool how such small sensors can locally run a ML model and just send what's important.
Imagine having 1000 devices — each running a tiny model on its own, detecting patterns or anomalies and sending only relevant data. That saves bandwidth and makes the system faster and more efficient.

So instead of having some big computing server doing everything, some things can just be done simpler and smarter — directly on the edge device.